

28.mobilenet模型代码封装

1. 目录结构



rk3588-mobilenet/

```
|—— CMakeLists.txt
|—— 3rdparty/
|   |—— opencv/
|—— images/
|—— librknn_api/
|   |—— include/
|   |—— aarch64/
|—— src/
|   |—— engine/
|       |—— rknn_engine.cpp
|       |—— rknn_engine.h
|   |—— process/
|       |—— preprocess.cpp
|       |—— postprocess.cpp
|       |—— postprocess.h
|   |—— mobilenet.cpp
|—— weights/
|—— images/
|—— utils/
    |—— engine_helper.h
    |—— logging.h
```

2. 源码分析

2.1 入口函数

代码块

```
1  // 导入相关库
2  #include <opencv2/opencv.hpp>
3
4  #include "engine/engine.h"
5  #include "process/preprocess.h"
6  #include "process/postprocess.h"
7  #include "utils/logging.h"
8
9  int main(int argc, char **argv)
10 {
11     // 模型文件路径
12     const char *model_file = argv[1];
13     // 图片文件路径
14     const char *img_file = argv[2];
15
16     // ===== 1. 加载引擎 =====
17     // 使用shared_ptr智能指针创建引擎，类型是接口中定义的NNEngine，但是实际上是
    RKEngine
18     std::shared_ptr<NNEngine> engine = CreateRKNNEngine();
19
20     // 加载模型文件、初始化rknn context、获取rknn版本信息、获取输入输出张量的信息
21     engine->LoadModelFile(model_file);
22
23     // 获取输入输出张量的形状
24     auto input_attrs = engine->GetInputShapes();
25     auto output_attrs = engine->GetOutputShapes();
26
27     NN_LOG_INFO("processing input data");
28
29     uint32_t height = input_attrs[0].dims[1]; // 根据输入属性查询输入张量的高度（参
    考: dims=[1, 224, 224, 3]）
30     uint32_t width = input_attrs[0].dims[2]; // 宽度
31
32     // ===== 2. 读取、预处理图片 =====
33     // 加载图片
34     cv::Mat img = cv::imread(img_file);
35     // 预处理 bgr->rgb, resize
36     cv::Mat img_resized;
37     imgPreprocess(img, img_resized, width, height);
38
39     // ===== 3. 设置输入、输出属性 =====
40     // 输入数据
41     tensor_data_s input;
```

```

42     input.attr.n_dims = 4;
43     input.attr.dims[0] = 1;
44     input.attr.dims[1] = height;
45     input.attr.dims[2] = width;
46     input.attr.dims[3] = 3;
47     input.attr.size = 3 * height * width;
48     input.attr.type = NN_TENSOR_UINT8;
49     input.attr.layout = NN_TENSOR_NHWC;
50     input.data = malloc(input.attr.size * sizeof(uint8_t));
51     // 将图片数据拷贝到input.data中
52     memcpy(input.data, img_resized.data, input.attr.size);
53     // 将input追加到inputs中
54     std::vector<tensor_data_s> inputs;
55     inputs.push_back(input);
56     NN_LOG_DEBUG("done preprocessing input data");
57
58     // 输出数据
59     std::vector<tensor_data_s> outputs;
60     tensor_data_s output;
61     output.data = malloc(output_attrs[0].n_elems * sizeof(float));
62     outputs.push_back(output);
63
64     // ===== 4. 推理 =====
65     NN_LOG_INFO("running...");
66     engine->Run(inputs, outputs, true);
67
68     // ===== 5. 后处理 =====
69     uint32_t MaxClass[5];
70     float fMaxProb[5];
71     float *buffer = (float *)output.data;
72     uint32_t sz = outputs[0].attr.size / 4;
73
74     NN_LOG_DEBUG("output size: %d", sz);
75
76     get_top(buffer, fMaxProb, MaxClass, sz, 5);
77
78     printf(" --- Top5 ---\n");
79     for (int i = 0; i < 5; i++)
80     {
81         printf("%3d: %.6f\n", MaxClass[i], fMaxProb[i]);
82     }
83     return 0;
84 }
85

```

2.2 engine目录

2.2.1 engine.h

定义一个纯虚基类 NNEngine。规定所有“神经网络推理引擎”必须实现哪些功能。

- 上层代码只依赖 NNEngine 接口
- 底层实现（RKNN/TFLite）可随时替换
- 解耦 + 可扩展 + 易测试

代码块

```
1  // 接口定义
2
3  #ifndef RK3588_DEMO_ENGINE_H
4  #define RK3588_DEMO_ENGINE_H
5
6  #include "types/error.h"
7  #include "types/datatype.h"
8
9  #include <vector>
10 #include <memory>
11
12 class NNEngine
13 {
14 public:
15     // 这里全部使用纯虚函数 (=0)，作用是将NNEngine定义为一个抽象类，不能实例化，只能作为基类使用
16     // 具体实现需要在子类中实现，这里的实现只是为了定义接口
17     // 用这种方式实现封装，可以使得不同的引擎的接口一致，方便使用；也可以隐藏不同引擎的实现细节，方便维护
18     virtual ~NNEngine(){};
19
20     // 析构函数
21     virtual nn_error_e LoadModelFile(const char *model_file) = 0;
22     // 加载模型文件，=0表示纯虚函数，必须在子类中实现
23     virtual const std::vector<tensor_attr_s> &GetInputShapes() = 0;
24     // 获取输入张量的形状
25     virtual const std::vector<tensor_attr_s> &GetOutputShapes() = 0;
26     // 获取输出张量的形状
27     virtual nn_error_e Run(std::vector<tensor_data_s> &inputs,
28         std::vector<tensor_data_s> &outpus, bool want_float) = 0; // 运行模型
29 };
30
31 std::shared_ptr<NNEngine> CreateRKNNEngine(); // 创建RKNN引擎
32
33 #endif // RK3588_DEMO_ENGINE_H
34
```

2.2.2 rknn_engine.h

声明 RKNN 引擎的具体类。继承自 NNEngine，承诺实现所有纯虚函数。

代码块

```
1  // 继承自NNEngine，实现NNEngine的接口
2
3  #ifndef RK3588_DEMO_RKNN_ENGINE_H
4  #define RK3588_DEMO_RKNN_ENGINE_H
5
6  #include "engine.h"
7
8  #include <vector>
9
10 #include <rknn_api.h>
11
12 // 继承自NNEngine，实现NNEngine的接口
13 class RKEngine : public NNEngine
14 {
15 public:
16     RKEngine() : rknn_ctx_(0), ctx_created_(false), input_num_(0),
17     output_num_(0){}; // 构造函数，初始化
18     ~RKEngine() override;
19     // 析构函数
20
21     nn_error_e LoadModelFile(const char *model_file) override; // 加载模型文件
22     const std::vector<tensor_attr_s> &GetInputShapes() override; // 获取输入张量的形状
23     const std::vector<tensor_attr_s> &GetOutputShapes() override; // 获取输出张量的形状
24     nn_error_e Run(std::vector<tensor_data_s> &inputs,
25     std::vector<tensor_data_s> &outputs, bool want_float) override; // 运行模型
26
27 private:
28     // rknn context
29     rknn_context rknn_ctx_; // rknn context
30     bool ctx_created_; // rknn context是否创建
31
32     uint32_t input_num_; // 输入的数量
33     uint32_t output_num_; // 输出的数量
34
35     std::vector<tensor_attr_s> in_shapes_; // 输入张量的形状
36     std::vector<tensor_attr_s> out_shapes_; // 输出张量的形状
37 };
38
```

```
37 #endif // RK3588_DEMO_RKNN_ENGINE_H
38
```

2.2.3 rknn_engine.cpp

代码块

```
1 // rknn_engine.h的实现
2
3 #include "rknn_engine.h"
4
5 #include <string.h>
6
7 #include "utils/engine_helper.h"
8 #include "utils/logging.h"
9
10 static const int g_max_io_num = 10; // 最大输入输出张量的数量
11
12 /**
13  * @brief 加载模型文件、初始化rknn context、获取rknn版本信息、获取输入输出张量的信息
14  * @param model_file 模型文件路径
15  * @return nn_error_e 错误码
16  */
17 nn_error_e RKEngine::LoadModelFile(const char *model_file)
18 {
19     int model_len = 0; // 模型文件大小
20     auto model = load_model(model_file, &model_len); // 加载模型文件
21     if (model == nullptr)
22     {
23         NN_LOG_ERROR("load model file %s fail!", model_file);
24         return NN_LOAD_MODEL_FAIL; // 返回错误码：加载模型文件失败
25     }
26     int ret = rknn_init(&rknn_ctx_, model, model_len, 0, NULL); // 初始化rknn
context
27     if (ret < 0)
28     {
29         NN_LOG_ERROR("rknn_init fail! ret=%d", ret);
30         return NN_RKNN_INIT_FAIL; // 返回错误码：初始化rknn context失败
31     }
32     // 打印初始化成功信息
33     NN_LOG_INFO("rknn_init success!");
34     ctx_created_ = true;
35
36     // 获取rknn版本信息
37     rknn_sdk_version version;
```

```

38     ret = rknn_query(rknn_ctx_, RKNN_QUERY_SDK_VERSION, &version,
sizeof(rknn_sdk_version));
39     if (ret < 0)
40     {
41         NN_LOG_ERROR("rknn_query fail! ret=%d", ret);
42         return NN_RKNN_QUERY_FAIL;
43     }
44     // 打印rknn版本信息
45     NN_LOG_INFO("RKNN API version: %s", version.api_version);
46     NN_LOG_INFO("RKNN Driver version: %s", version.drv_version);
47
48     // 获取输入输出个数
49     rknn_input_output_num io_num;
50     ret = rknn_query(rknn_ctx_, RKNN_QUERY_IN_OUT_NUM, &io_num,
sizeof(io_num));
51     if (ret != RKNN_SUCC)
52     {
53         NN_LOG_ERROR("rknn_query fail! ret=%d", ret);
54         return NN_RKNN_QUERY_FAIL;
55     }
56     NN_LOG_INFO("model input num: %d, output num: %d", io_num.n_input,
io_num.n_output);
57
58     // 保存输入输出个数
59     input_num_ = io_num.n_input;
60     output_num_ = io_num.n_output;
61
62     // 输入属性
63     NN_LOG_INFO("input tensors:");
64     rknn_tensor_attr input_attrs[io_num.n_input];
65     memset(input_attrs, 0, sizeof(input_attrs));
66     for (int i = 0; i < io_num.n_input; i++)
67     {
68         input_attrs[i].index = i;
69         ret = rknn_query(rknn_ctx_, RKNN_QUERY_INPUT_ATTR, &(input_attrs[i]),
sizeof(rknn_tensor_attr));
70         if (ret != RKNN_SUCC)
71         {
72             NN_LOG_ERROR("rknn_query fail! ret=%d", ret);
73             return NN_RKNN_QUERY_FAIL;
74         }
75         print_tensor_attr(&(input_attrs[i]));
76         // set input_shapes_
77         in_shapes_.push_back(rknn_tensor_attr_convert(input_attrs[i]));
78     }
79
80     // 输出属性

```

```

81     NN_LOG_INFO("output tensors:");
82     rknn_tensor_attr output_attrs[io_num.n_output];
83     memset(output_attrs, 0, sizeof(output_attrs));
84     for (int i = 0; i < io_num.n_output; i++)
85     {
86         output_attrs[i].index = i;
87         ret = rknn_query(rknn_ctx_, RKNN_QUERY_OUTPUT_ATTR, &
(output_attrs[i]), sizeof(rknn_tensor_attr));
88         if (ret != RKNN_SUCC)
89         {
90             NN_LOG_ERROR("rknn_query fail! ret=%d", ret);
91             return NN_RKNN_QUERY_FAIL;
92         }
93         print_tensor_attr(&(output_attrs[i]));
94         // set output_shapes_
95         out_shapes_.push_back(rknn_tensor_attr_convert(output_attrs[i]));
96     }
97
98     return NN_SUCCESS;
99 }
100
101 // 获取输入张量的形状
102 const std::vector<tensor_attr_s> &RKEngine::GetInputShapes()
103 {
104     return in_shapes_;
105 }
106
107 // 获取输出张量的形状
108 const std::vector<tensor_attr_s> &RKEngine::GetOutputShapes()
109 {
110     return out_shapes_;
111 }
112
113 /**
114  * @brief 运行模型，获得推理结果
115  * @param inputs 输入张量
116  * @param outputs 输出张量
117  * @param want_float 是否需要float类型的输出
118  * @return nn_error_e 错误码
119  */
120 nn_error_e RKEngine::Run(std::vector<tensor_data_s> &inputs,
std::vector<tensor_data_s> &outputs, bool want_float)
121 {
122     // 检查输入输出张量的数量是否匹配
123     if (inputs.size() != input_num_)
124     {

```

```

125         NN_LOG_ERROR("inputs num not match! inputs.size()=%ld, input_num_=%d",
inputs.size(), input_num_);
126         return NN_IO_NUM_NOT_MATCH;
127     }
128     if (outputs.size() != output_num_)
129     {
130         NN_LOG_ERROR("outputs num not match! outputs.size()=%ld,
output_num_=%d", outputs.size(), output_num_);
131         return NN_IO_NUM_NOT_MATCH;
132     }
133
134     // 设置rknn inputs
135     rknn_input rknn_inputs[g_max_io_num];
136     for (int i = 0; i < inputs.size(); i++)
137     {
138         // 将自定义的tensor_data_s转换为rknn_input
139         rknn_inputs[i] = tensor_data_to_rknn_input(inputs[i]);
140     }
141     int ret = rknn_inputs_set(rknn_ctx_, (uint32_t)inputs.size(), rknn_inputs);
142     if (ret < 0)
143     {
144         NN_LOG_ERROR("rknn_inputs_set fail! ret=%d", ret);
145         return NN_RKNN_INPUT_SET_FAIL;
146     }
147
148     // 推理
149     NN_LOG_DEBUG("rknn running...");
150     ret = rknn_run(rknn_ctx_, nullptr);
151     if (ret < 0)
152     {
153         NN_LOG_ERROR("rknn_run fail! ret=%d", ret);
154         return NN_RKNN_RUNTIME_ERROR;
155     }
156
157     // 获得输出
158     rknn_output rknn_outputs[g_max_io_num];
159     memset(rknn_outputs, 0, sizeof(rknn_outputs));
160     for (int i = 0; i < output_num_; ++i)
161     {
162         rknn_outputs[i].want_float = want_float ? 1 : 0;
163     }
164     ret = rknn_outputs_get(rknn_ctx_, output_num_, rknn_outputs, NULL);
165     if (ret < 0)
166     {
167         printf("rknn_outputs_get fail! ret=%d\n", ret);
168         NN_LOG_ERROR("rknn_outputs_get fail! ret=%d", ret);
169         return NN_RKNN_OUTPUT_GET_FAIL;

```

```

170     }
171
172     NN_LOG_DEBUG("output num: %d", output_num_);
173     // copy rknn outputs to tensor_data_s
174     for (int i = 0; i < output_num_; ++i)
175     {
176         // 将rknn_output转换为自定义的tensor_data_s
177         rknn_output_to_tensor_data(rknn_outputs[i], outputs[i]);
178         NN_LOG_DEBUG("output[%d] size=%d", i, outputs[i].attr.size);
179     }
180     return NN_SUCCESS;
181 }
182
183 // 析构函数
184 RKEngine::~RKEngine()
185 {
186     if (ctx_created_)
187     {
188         rknn_destroy(rknn_ctx_);
189         NN_LOG_INFO("rknn context destroyed!");
190     }
191 }
192
193 // 创建RKNN引擎
194 std::shared_ptr<NNEngine> CreateRKNNEngine()
195 {
196     return std::make_shared<RKEngine>();
197 }
198

```

(1) 全局常量

代码块

```

1 static const int g_max_io_num = 10; // 最大支持 10 个输入或输出

```

避免动态分配，提升嵌入式性能（栈上数组）

(2) LoadModelFile() —— 加载模型并初始化

流程：

1. 调用 load_model() 读取 .rknn 文件到内存
2. 调用 rknn_init() 初始化 NPU 上下文
3. 查询 SDK 和驱动版本（用于调试）

4. 查询输入/输出数量

5. 遍历每个输入/输出，获取其属性（shape/type/layout/量化参数）

(3) GetInputShapes() / GetOutputShapes()

获取输入/输出张量的形状

(4) Run() —— 核心推理函数

流程：

1. 校验输入/输出数量 是否匹配模型

2. 转换输入：std::vector<tensor_data_s> → rknn_input[]

3. 调用 rknn_inputs_set() 设置输入

4. 调用 rknn_run() 执行推理

5. 调用 rknn_outputs_get() 获取结果

6. 转换输出：rknn_output[] → std::vector<tensor_data_s>& outputs

7. 返回成功/错误码

(5) 析构函数 ~RKEngine()

(6) 工厂函数 CreateRKNNEngine()

2.3 process目录

2.3.1 preprocess.cpp

这个代码很简单就是把BGR转为RGB

代码块

```
1  // 预处理
2
3  #include "preprocess.h"
4
5  #include "utils/logging.h"
6
7  void imgPreprocess(const cv::Mat &img, cv::Mat &img_resized, uint32_t width,
8  uint32_t height)
9  {
10     // img has to be 3 channels
11     if (img.channels() != 3)
12     {
13         NN_LOG_ERROR("img has to be 3 channels");
14         exit(-1);
15     }
```

```

16     cv::Mat img_rgb;
17     // bgr to rgb
18     cv::cvtColor(img, img_rgb, cv::COLOR_BGR2RGB);
19     // resize img
20     cv::resize(img_rgb, img_resized, cv::Size(width, height), 0, 0,
cv::INTER_LINEAR);
21
22 }

```

2.3.2 postprocess.cpp

进行了一个排序

代码块

```

1
2  #include "postprocess.h"
3
4  #include <string.h>
5  #include <stdlib.h>
6
7  #include <algorithm>
8
9  #include "utils/logging.h"
10
11 int get_top(float* pfProb, float* pfMaxProb, uint32_t* pMaxClass, uint32_t
outputCount, uint32_t topNum) {
12     uint32_t i, j;
13
14     #define MAX_TOP_NUM 20
15     if (topNum > MAX_TOP_NUM)
16         return 0;
17
18     memset(pfMaxProb, 0, sizeof(float) * topNum);
19     memset(pMaxClass, 0xff, sizeof(float) * topNum);
20
21     for (j = 0; j < topNum; j++) {
22         for (i = 0; i < outputCount; i++) {
23             if ((i == *(pMaxClass + 0)) || (i == *(pMaxClass + 1)) || (i == *
(pMaxClass + 2)) || (i == *(pMaxClass + 3)) ||
24                 (i == *(pMaxClass + 4))) {
25                 continue;
26             }
27
28             if (pfProb[i] > *(pfMaxProb + j)) {
29                 *(pfMaxProb + j) = pfProb[i];
30                 *(pMaxClass + j) = i;

```

```

31         }
32     }
33 }
34
35     return 1;
36 }
37

```

2.4 types目录

2.4.1 error.h

代码块

```

1  // 错误码定义
2
3  #ifndef RK3588_DEMO_ERROR_H
4  #define RK3588_DEMO_ERROR_H
5
6  typedef enum
7  {
8      NN_SUCCESS = 0,                // 成功
9      NN_LOAD_MODEL_FAIL = -1,       // 加载模型失败
10     NN_RKNN_INIT_FAIL = -2,        // rknn初始化失败
11     NN_RKNN_QUERY_FAIL = -3,       // rknn查询失败
12     NN_RKNN_INPUT_SET_FAIL = -4,   // rknn设置输入数据失败
13     NN_RKNN_RUNTIME_ERROR = -5,    // rknn运行时错误
14     NN_IO_NUM_NOT_MATCH = -6,      // 输入输出数量不匹配
15     NN_RKNN_OUTPUT_GET_FAIL = -7,  // rknn获取输出数据失败
16     NN_RKNN_INPUT_ATTR_ERROR = -8, // rknn输入数据属性错误
17     NN_RKNN_OUTPUT_ATTR_ERROR = -9, // rknn输出数据属性错误
18     NN_RKNN_MODEL_NOT_LOAD = -10,  // rknn模型未加载
19 } nn_error_e;
20
21 #endif // RK3588_DEMO_ERROR_H
22

```

2.4.2 datatype.h

代码块

```

1  // 定义数据类型
2
3  #ifndef RK3588_DEMO_DATATYPE_H
4  #define RK3588_DEMO_DATATYPE_H

```

```

5
6 #include <stdint.h>
7 #include <stdlib.h>
8
9 #include "utils/logging.h"
10 #include "types/error.h"
11
12 typedef enum _tensor_layout // 张量内存布局
13 {
14     NN_TENSOR_LAYOUT_UNKNOWN = 0, // 未知
15     NN_TENSOR_NCHW = 1, // 通道优先 (适合 GPU 计算)
16     NN_TENSOR_NHWC = 2, // 空间优先 (适合 CPU/OpenCV)
17     NN_TENSOR_OTHER = 3, // 其他非标准布局 (预留扩展)
18 } tensor_layout_e;
19
20 typedef enum _tensor_datatype // 数据类型
21 {
22     NN_TENSOR_INT8 = 1, // 有符号 8 位整数 (-128 ~ 127)
23     NN_TENSOR_UINT8 = 2, // 无符号 8 位整数 (0 ~ 255)
24     NN_TENSOR_FLOAT = 3, // 32 位浮点数 (标准 float)
25     NN_TENSOR_FLOAT16 = 4, // 16 位半精度浮点 (节省内存, 部分 NPU 支持)
26 } tensor_datatype_e;
27
28 static const int g_max_num_dims = 4; // 限制张量最大维度为 4
29
30 typedef struct
31 {
32     uint32_t index; // 张量索引 (第几个输入/输出, 从 0 开始)
33     uint32_t n_dims; // 实际维度数 (≤4, 如 4 表示 [N,C,H,W])
34     uint32_t dims[g_max_num_dims]; // 各维度大小 (固定长度数组, 未用维度填 0)
35     uint32_t n_elems; // 总元素个数 = dims[0] * dims[1] * ...
36     uint32_t size; // 总字节数 = n_elems * sizeof(type)
37     tensor_datatype_e type; // 数据类型 (UINT8 / FLOAT 等)
38     tensor_layout_e layout; // 内存布局 (NCHW / NHWC)
39     int32_t zp; // 量化零点 (Zero Point), 仅用于 INT8/UINT8
40     float scale; // 量化缩放因子 (Scale), 仅用于量化模型
41 } tensor_attr_s;
42
43 typedef struct
44 {
45     tensor_attr_s attr; // 张量的元信息 (形状、类型等)
46     void *data; // 指向实际数据的指针 (如图像像素、模型输出概率)
47 } tensor_data_s;
48
49 static size_t nn_tensor_type_to_size(tensor_datatype_e type) // 获取数据类型的字
    节大小
50 {

```

```
51     switch (type)
52     {
53     case NN_TENSOR_INT8:
54         return sizeof(int8_t);
55     case NN_TENSOR_UINT8:
56         return sizeof(uint8_t);
57     case NN_TENSOR_FLOAT:
58         return sizeof(float);
59     case NN_TENSOR_FLOAT16:
60         return sizeof(uint16_t);
61     default:
62         NN_LOG_ERROR("unsupported tensor type");
63         exit(-1);
64     }
65 }
66
67 // 将任意模型输入的 tensor_attr_s 转换为 OpenCV 图像兼容的格式 (NHWC + UINT8)
68 static void nn_tensor_attr_to_cvimg_input_data(const tensor_attr_s &attr,
69 tensor_data_s &data)
70 {
71     if (attr.n_dims != 4)
72     {
73         NN_LOG_ERROR("unsupported input dims");
74         exit(-1);
75     }
76     data.attr.n_dims = attr.n_dims;
77     data.attr.index = 0;
78     data.attr.type = NN_TENSOR_UINT8;
79     data.attr.layout = NN_TENSOR_NHWC;
80     if (attr.layout == NN_TENSOR_NCHW)
81     {
82         data.attr.dims[0] = attr.dims[0];
83         data.attr.dims[1] = attr.dims[2];
84         data.attr.dims[2] = attr.dims[3];
85         data.attr.dims[3] = attr.dims[1];
86     }
87     else if (attr.layout == NN_TENSOR_NHWC)
88     {
89         data.attr.dims[0] = attr.dims[0];
90         data.attr.dims[1] = attr.dims[1];
91         data.attr.dims[2] = attr.dims[2];
92         data.attr.dims[3] = attr.dims[3];
93     }
94     else
95     {
96         NN_LOG_ERROR("unsupported input layout");
97         exit(-1);
98     }
99 }
```

```

97     }
98     // multiply all dims
99     data.attr.n_elems = data.attr.dims[0] * data.attr.dims[1] *
100                        data.attr.dims[2] * data.attr.dims[3];
101     data.attr.size = data.attr.n_elems * sizeof(uint8_t);
102 }
103
104 #endif // RK3588_DEMO_DATATYPE_H
105

```

2.5 utils目录

2.5.1 logging.h

代码块

```

1
2 #ifndef RK3588_DEMO_LOGGING_H
3 #define RK3588_DEMO_LOGGING_H
4
5 // a logging wrapper so it can be easily replaced
6 #include <stdio.h>
7
8 // log level from low to high
9 // 0: no log
10 // 1: error
11 // 2: error, warning
12 // 3: error, warning, info
13 // 4: error, warning, info, debug
14 static int32_t g_log_level = 4;
15
16 // a printf wrapper so the msg can be formatted with %d %s, etc.
17
18 #define NN_LOG_ERROR(...) \
19     do \
20     { \
21         if (g_log_level >= 1) \
22         { \
23             printf("[NN_ERROR] "); \
24             printf(__VA_ARGS__); \
25             printf("\n"); \
26         } \
27     } while (0)
28

```

```

29  #define NN_LOG_WARNING(...) \
30      do \
31      { \
32          if (g_log_level >= 2) \
33          { \
34              printf("[NN_WARNING] "); \
35              printf(__VA_ARGS__); \
36              printf("\n"); \
37          } \
38      } while (0)
39
40  #define NN_LOG_INFO(...) \
41      do \
42      { \
43          if (g_log_level >= 3) \
44          { \
45              printf("[NN_INFO] "); \
46              printf(__VA_ARGS__); \
47              printf("\n"); \
48          } \
49      } while (0)
50
51  #define NN_LOG_DEBUG(...) \
52      do \
53      { \
54          if (g_log_level >= 4) \
55          { \
56              printf("[NN_DEBUG] "); \
57              printf(__VA_ARGS__); \
58              printf("\n"); \
59          } \
60      } while (0)
61
62  #endif // RK3588_DEMO_LOGGING_H
63

```

2.5.2 engine_helper.h

代码块

```

1  // 辅助函数
2
3  #ifndef RK3588_DEMO_ENGINE_HELPER_H
4  #define RK3588_DEMO_ENGINE_HELPER_H
5
6  #include <fstream>

```

```

7
8 #include <string.h>
9
10 #include <vector>
11
12 #include <rknn_api.h>
13
14 #include "utils/logging.h"
15 #include "types/datatype.h"
16
17 /**
18  * @brief 加载模型文件
19  * @param filename 模型文件路径
20  * @param model_size 模型文件大小，会在函数内部赋值
21  * @return unsigned char* 模型文件数据
22  */
23 static unsigned char *load_model(const char *filename, int *model_size)
24 {
25     FILE *fp = fopen(filename, "rb");
26     if (fp == nullptr)
27     {
28         NN_LOG_ERROR("fopen %s fail!", filename);
29         return nullptr;
30     }
31     fseek(fp, 0, SEEK_END);
32     int model_len = ftell(fp);
33     unsigned char *model = (unsigned char *)malloc(model_len);
34     fseek(fp, 0, SEEK_SET);
35     if (model_len != fread(model, 1, model_len, fp))
36     {
37         NN_LOG_ERROR("fread %s fail!", filename);
38         free(model);
39         return nullptr;
40     }
41     *model_size = model_len;
42     if (fp)
43     {
44         fclose(fp);
45     }
46     return model;
47 }
48
49 static void print_tensor_attr(rknn_tensor_attr *attr)
50 {
51     NN_LOG_INFO("  index=%d, name=%s, n_dims=%d, dims=[%d, %d, %d, %d],
52     n_elems=%d, size=%d, fmt=%s, type=%s, qnt_type=%s, "
53     "zp=%d, scale=%f",

```

```

53         attr->index, attr->name, attr->n_dims, attr->dims[0], attr-
>dims[1], attr->dims[2], attr->dims[3],
54         attr->n_elems, attr->size, get_format_string(attr->fmt),
get_type_string(attr->type),
55         get_qnt_type_string(attr->qnt_type), attr->zp, attr->scale);
56     }
57
58     static tensor_layout_e rknn_layout_convert(rknn_tensor_format fmt)
59     {
60         switch (fmt)
61         {
62             case RKNN_TENSOR_NCHW:
63                 return NN_TENSOR_NCHW;
64             case RKNN_TENSOR_NHWC:
65                 return NN_TENSOR_NHWC;
66             default:
67                 return NN_TENSOR_OTHER;
68         }
69     }
70
71     static rknn_tensor_format rknn_layout_convert(tensor_layout_e fmt)
72     {
73         switch (fmt)
74         {
75             case NN_TENSOR_NCHW:
76                 return RKNN_TENSOR_NCHW;
77             case NN_TENSOR_NHWC:
78                 return RKNN_TENSOR_NHWC;
79             default:
80                 NN_LOG_ERROR("unsupported nn layout: %d\n", fmt);
81                 // exit program
82                 exit(1);
83         }
84     }
85
86     static rknn_tensor_type rknn_type_convert(tensor_datatype_e type)
87     {
88         switch (type)
89         {
90             case NN_TENSOR_UINT8:
91                 return RKNN_TENSOR_UINT8;
92             case NN_TENSOR_FLOAT:
93                 return RKNN_TENSOR_FLOAT32;
94             default:
95                 NN_LOG_ERROR("unsupported nn type: %d\n", type);
96                 // exit program
97                 exit(1);

```

```

98     }
99 }
100
101 static tensor_datatype_e rknn_type_convert(rknn_tensor_type type)
102 {
103     switch (type)
104     {
105     case RKNN_TENSOR_UINT8:
106         return NN_TENSOR_UINT8;
107     case RKNN_TENSOR_FLOAT32:
108         return NN_TENSOR_FLOAT;
109     case RKNN_TENSOR_INT8:
110         return NN_TENSOR_INT8;
111     case RKNN_TENSOR_FLOAT16:
112         return NN_TENSOR_FLOAT16;
113     default:
114         NN_LOG_ERROR("unsupported rknn type: %d\n", type);
115         // exit program
116         exit(1);
117     }
118 }
119
120 static tensor_attr_s rknn_tensor_attr_convert(const rknn_tensor_attr &attr)
121 {
122     tensor_attr_s shape;
123     shape.n_dims = attr.n_dims;
124     shape.index = attr.index;
125     for (int i = 0; i < attr.n_dims; ++i)
126     {
127         shape.dims[i] = attr.dims[i];
128     }
129     shape.size = attr.size;
130     shape.n_elems = attr.n_elems;
131     // set layout
132     shape.layout = rknn_layout_convert(attr.fmt);
133     shape.type = rknn_type_convert(attr.type);
134     shape.zp = attr.zp;
135     shape.scale = attr.scale;
136     return shape;
137 }
138
139 static rknn_input tensor_data_to_rknn_input(const tensor_data_s &data)
140 {
141     rknn_input input;
142     memset(&input, 0, sizeof(input));
143     // set default not passthrough
144     input.index = data.attr.index;

```

```

145     input.type = rknn_type_convert(data.attr.type);
146     input.size = data.attr.size;
147     input.fmt = rknn_layout_convert(data.attr.layout);
148     input.buf = data.data;
149     return input;
150 }
151
152 static void rknn_output_to_tensor_data(const rknn_output &output,
    tensor_data_s &data)
153 {
154     data.attr.index = output.index;
155     data.attr.size = output.size;
156     NN_LOG_DEBUG("output size: %d", output.size);
157     NN_LOG_DEBUG("output want_float: %d", output.want_float);
158     memcpy(data.data, output.buf, output.size);
159 }
160
161 #endif // RK3588_DEMO_ENGINE_HELPER_H
162

```

3. 代码编译

直接执行./build.sh即可，build.sh的内容如下：

代码块

```

1  #!/bin/bash
2  set -e
3  export PATH=/opt/tool_chain/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-
    gnu/bin:$PATH
4  rm -rf build
5
6  cmake -S . -B build \
7      -DCMAKE_SYSTEM_NAME=Linux \
8      -DCMAKE_CXX_COMPILER=aarch64-rockchip1031-linux-gnu-g++
9
10 cmake --build build -j4
11 cmake --install build
12 echo "✅ 打包完成！部署目录：$PWD/install"

```

然后把编译出来的install直接传输到开发板上面就行。

查看结果

```
root@topeet:/userdata/rk3588-mobilenet-v2# ./rknn_engine_test ./model/mobilenet_v1.rknn ./model/dog_224x224.jpg
[NN_INFO] rknn_init success!
[NN_INFO] RKNN API version: 2.3.2 (429f97ae6b@2025-04-09T09:09:27)
[NN_INFO] RKNN Driver version: 0.9.8
[NN_INFO] model input num: 1, output num: 1
[NN_INFO] input tensors:
[NN_INFO]   index=0, name=input, n_dims=4, dims=[1, 224, 224, 3], n_elems=150528, size=150528, fmt=NHWC, type=INT8, qnt_type=AFFINE, zp=0, scale=0.007812
[NN_INFO] output tensors:
[NN_INFO]   index=0, name=MobilenetV1/Predictions/Reshape_1, n_dims=2, dims=[1, 1001, 0, 0], n_elems=1001, size=2002, fmt=UNDEFINED, type=FP16, qnt_type=AFFINE, zp=0, scale=1.000000
[NN_INFO] processing input data
[NN_DEBUG] done preprocessing input data
[NN_INFO] running...
[NN_DEBUG] rknn running...
[NN_DEBUG] output num: 1
[NN_DEBUG] output size: 4004
[NN_DEBUG] output want_float: 1
[NN_DEBUG] output[0] size=4004
[NN_DEBUG] output size: 1001
--- Top5 ---
156: 0.884766
155: 0.054016
205: 0.003677
284: 0.002974
285: 0.000189
[NN_INFO] rknn context destroyed!
root@topeet:/userdata/rk3588-mobilenet-v2#
```